

EECS 482

Introduction to operating systems

Reader/writer locks; semaphores

Peter Chen
University of Michigan

Reader-writer locks

- Example: multiple threads would like to read or write Wolverine Access course catalog
- Threads need to acquire lock to read or write shared data

Student:

lock()

browse course catalog

unlock()

Administrator:

lock()

change course catalog

unlock()

- How to safely allow more concurrency?

How to safely allow more concurrency?

- Threads disclose whether they intend to read or write the shared data

Student:

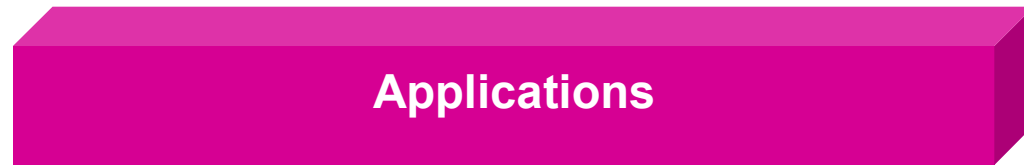
```
lock() readerLock()  
browse course catalog  
unlock() readerUnlock()
```

Administrator:

```
lock() writerLock()  
change course catalog  
unlock() writerUnlock()
```

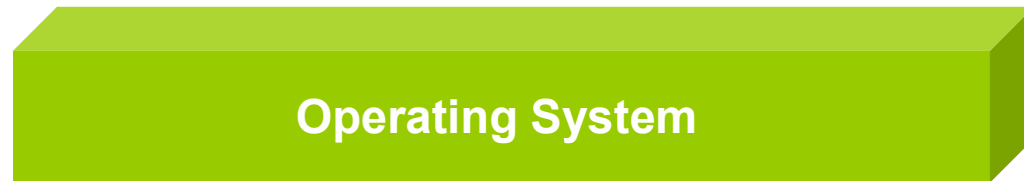
- Allow **multiple concurrent readers**, if no threads are writing data
- Allow a **single writer**, if no other threads are reading or writing

Another level of abstraction



Applications

Even higher-level
synchronization operations
Concurrent programs
(readerLock, readerUnlock,
writerLock, writerUnlock)



Operating System

Higher-level synchronization
operations
(lock, monitor, semaphore)



Hardware

Atomic operations
(load/store, interrupt enable/
disable, test&set)

Reader-writer lock with monitors

- Step 1: What shared data is needed to implement readerLock, readerUnlock, writerLock, writerUnlock?
- Step 2: Assign locks to shared state:
- Step 3: List the before-after conditions and assign a condition variable for each condition

Reader-writer lock with monitors

```
int numReaders, numWriters
```

```
readerLock () {  
    numReaders++  
}
```

```
writerLock() {  
    numWriters++  
}
```

```
readerUnlock() {  
    numReaders--  
}
```

```
writerUnlock() {  
    numWriters--  
}
```

Reader-writer lock with monitors

- What will happen if a writer finishes and there are several waiting readers and writers?
- How long will a writer wait?
- How to give priority to a waiting writer?

Project 1: Deadlocks

- Where can a thread block?
- Add print statements before/after each blocking call to determine where each thread is
- Who should be running? Why is that thread not running?

Project 1: Problematic program patterns

Pattern 1:

```
while (condition) {  
    cv.signal()  
    cv.wait(mutex1)  
}
```

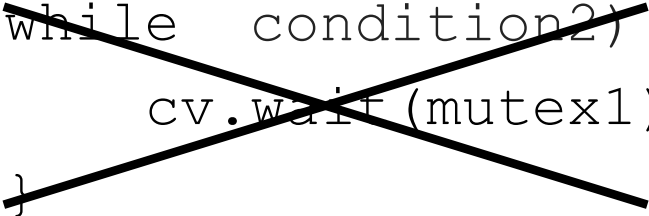
Pattern 2:

```
while (condition) {  
    mutex2.unlock()  
     cv.wait(mutex1)  
    mutex2.lock()  
}
```

Project 1: Problematic program patterns

Pattern 3:

```
while (condition1 ||
      cv.wait(mutex1)
)
while condition2) {
    cv.wait(mutex1)
}
```



Review: writing concurrent programs

- Multiple threads work on shared data
 - Must synchronize to ensure correct results
- Synchronization
 - Two types: mutual exclusion, ordering
 - Difficult with low-level mechanism (e.g., atomic load, atomic store)
- Monitors
 - **Locks** for mutual exclusion
 - **Condition variables** for ordering constraints

Semaphores (sometimes called counting semaphores)

- Generalized lock/unlock
 - A **non-negative integer** (initialized to user-specified value)
 - **down()**: wait for semaphore value to become positive, then decrement semaphore value by 1

```
while (1) {
```

```
    if (value > 0) {
```

```
        value--
```

```
        break
```

```
    }
```

```
}
```

Atomic

- **up()**: increment semaphore value by 1

```
value++
```

Atomic

Special case: binary semaphore

- Semaphore value is 0 or 1
- up() atomically **sets** value to 1

Semaphores can enforce mutual exclusion and ordering

- Mutual exclusion

```
semaphore sem(1)
```

```
sem.down()
```

```
critical section
```

```
sem.up()
```

- Ordering constraints

- Example: ensure that Thread A's task is done before Thread B's task

```
semaphore sem(0)
```

Thread A

```
task A
```

```
sem.up()
```

Thread B

```
sem.down()
```

```
task B
```

Producer-consumer with semaphores

- As always, think about shared data, mutual exclusion, and before-after constraints
- Semaphores:
 - *mtx*: for exclusive access to coke machine
 - *fullSlots*: counts number of cokes in machine
 - *emptySlots*: counts number of empty slots in machine
- Before-after constraints
 - Consumer must wait if no cokes in machine
 - Producer must wait if machine is full

Producer-consumer with semaphores

- Why do we need different semaphores for fullSlots and emptySlots?
- What if there's 1 full slot, and multiple consumers call down() at the same time?

Producer-consumer with semaphores

```
semaphore mtx(1)           // mutual exclusion to shared data
semaphore fullSlots(0)      // number of full slots
semaphore emptySlots(MAX)  // number of empty slots
```

```
producer {
    mtx.down()

    emptySlots.down()

    // add coke to machine

    fullSlots.up()

    mtx.up()

}
```

```
consumer {
    mtx.down()

    fullSlots.down()

    // take coke out of machine

    emptySlots.up()

    mtx.up()

}
```

Producer-consumer with semaphores

```
semaphore mtx(1);           // mutual exclusion to shared data
semaphore fullSlots(0);     // number of full slots
semaphore emptySlots(MAX);  // number of empty slots
```

```
producer {
    mtx.down()

     emptySlots.down()

    // add coke to machine

    fullSlots.up()

    mtx.up()

}
```

```
consumer {
    mtx.down()

    fullSlots.down()

    // take coke out of machine

    emptySlots.up()

    mtx.up()

}
```

What's wrong with this solution?

Producer-consumer with semaphores

```
semaphore mtx(1);           // mutual exclusion to shared data
semaphore fullSlots(0);     // number of full slots
semaphore emptySlots(MAX);  // number of empty slots
```



```
producer {
    mtx.down()

    emptySlots.down()

    // add coke to machine

    fullSlots.up()

    mtx.up()

}
```

```
consumer {
    mtx.down()

    fullSlots.down()

    // take coke out of machine

    emptySlots.up()

    mtx.up()

}
```

Any concerns?

Producer-consumer with semaphores

- What about the state of the system between `emptySlots.down()` and `mtx.down()`?
- Does the order of `up()` matter?

What if producer called fullSlots.up() earlier?

```
semaphore mtx(1)           // mutual exclusion to shared data
semaphore fullSlots(0)     // number of full slots
semaphore emptySlots(MAX)  // number of empty slots
```

```
producer {
    emptySlots.down()

    
    mtx.down()

    // add coke to machine

    mtx.up()

    fullSlots.up()

}
```

```
consumer {
    fullSlots.down()

    mtx.down()

    // take coke out of machine

    mtx.up()

    emptySlots.up()

}
```

Monitors vs. semaphores

- Monitors
 - Locks for mutual exclusion
 - Condition variables for ordering
- Semaphores: one mechanism for both mutual exclusion and ordering
 - Elegantly minimal
 - Can be difficult to use

Mutual exclusion (locks vs. semaphores)

- lock = binary semaphore (initialized to 1)
- lock() = down()
- unlock() = up()

Mutex	Semaphore
<pre>mutex m m.lock() <critical section> m.unlock()</pre>	<pre>semaphore m(1) m.down() <critical section> m.up()</pre>

Ordering (condition variables vs. semaphores)

Condition variable	Semaphore
<code>while (condition) {wait(m)}</code>	<code>down()</code>
Waiting condition is in user code and uses user variables	Waiting condition is in semaphore code and uses semaphore's value
Waiting condition defined by user (more flexible)	Waiting condition defined by semaphore (wait if semaphore value==0)
User variables are protected by mutex	Semaphore's value is protected by semaphore's implementation of <code>up()</code> , <code>down()</code>
Must hold lock when calling wait	Must not hold "lock" (semaphore) when calling <code>down()</code> for ordering

Ordering (condition variables vs. semaphores)

Condition variable	Semaphore
No memory of past signals: <u>Thread A</u> wait() <u>Thread B</u> signal() What happens if wait(), then signal()? What happens if signal(), then wait()? Why is this ok?	Remembers past up calls <u>Thread A</u> down() <u>Thread B</u> up() What happens if down(), then up()? What happens if up(), then down()?